DAT151: Assignment 4 Report

Group: 5

Group Members: Soukup Jan, Fabienne Feilke

Date: March 20, 2026

This report was written after Monday's 16.3. review during lab.

Task 1: Triggers

Objective: As a normal database user (i.e. not root), create triggers on table TheTable so that all changes of TheTable will be logged in LogTable. Make sure all values before and after a change are logged.

Requirements:

- For DELETE, you must log the action DELETE, and the row before values.
- For INSERT, you must log the action INSERT, and the row after values.
- For UPDATE, you must log the action UPDATE, and both the row before and after values.
    - The LogTable must clearly identify the old and the new values.

Delivery:

- The DDL of the triggers
- The DDL of the tables
- Output that demonstrates the working of the triggers

Creating the Tables:

```
MariaDB [ffeilkedatabase]> CREATE TABLE AnotherTable (
    ->     ID int(11) NOT NULL,
    ->     Value varchar(50),
    ->     PRIMARY KEY (ID)
    -> ) WITH SYSTEM VERSIONING;
Query OK, 0 rows affected (0.014 sec)

MariaDB [ffeilkedatabase]> CREATE TABLE LogTable (
    ->     LogID INT AUTO_INCREMENT PRIMARY KEY,
    ->     ActionType VARCHAR(10),
    ->     RecordID INT,
    ->     Old_Value VARCHAR(50),
    ->     New_Value VARCHAR(50),
    ->     ChangeTime TIMESTAMP(6) DEFAULT CURRENT_TIMESTAMP(6)
    -> );
Query OK, 0 rows affected (0.012 sec)
```

)

The DDL of the Triggers:

```
MariaDB [ffeilkedatabase]> DELIMITER //
MariaDB [ffeilkedatabase]> CREATE TRIGGER trg_after_insert
    -> AFTER INSERT ON AnotherTable
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO LogTable (ActionType, RecordID, New_Value)
    ->     VALUES ('INSERT', NEW.ID, NEW.Value);
    -> END//
Query OK, 0 rows affected (0.011 sec)

MariaDB [ffeilkedatabase]>
MariaDB [ffeilkedatabase]>
MariaDB [ffeilkedatabase]> CREATE TRIGGER trg_after_update
    -> AFTER UPDATE ON AnotherTable
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO LogTable (ActionType, RecordID, Old_Value, New_V
    ->     VALUES ('UPDATE', NEW.ID, OLD.Value, NEW.Value);
    -> END//
Query OK, 0 rows affected (0.011 sec)

MariaDB [ffeilkedatabase]>
MariaDB [ffeilkedatabase]>
MariaDB [ffeilkedatabase]> CREATE TRIGGER trg_after_delete
    -> AFTER DELETE ON AnotherTable
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO LogTable (ActionType, RecordID, Old_Value)
    ->     VALUES ('DELETE', OLD.ID, OLD.Value);
    -> END//
Query OK, 0 rows affected (0.011 sec)
```

Demonstrate the working of the triggers:

```
MariaDB [ffeilkedatabase]> INSERT INTO AnotherTable (ID, Value) VALUES (10, 'First Version');
Query OK, 1 row affected (0.005 sec)

MariaDB [ffeilkedatabase]> UPDATE AnotherTable SET Value = 'Second Version' WHERE ID = 10;
Query OK, 1 row affected (0.004 sec)
Rows matched: 1  Changed: 1  Inserted: 1  Warnings: 0

MariaDB [ffeilkedatabase]> DELETE FROM AnotherTable WHERE ID = 10;
Query OK, 1 row affected (0.004 sec)

MariaDB [ffeilkedatabase]> SELECT ActionType, RecordID, Old_Value, New_Value FROM LogTable;
+------------+----------+----------------+----------------+
| ActionType | RecordID | Old_Value      | New_Value      |
+------------+----------+----------------+----------------+
| INSERT     |       10 | NULL           | First Version  |
| UPDATE     |       10 | First Version  | Second Version |
| DELETE     |       10 | Second Version | NULL           |
+------------+----------+----------------+----------------+
3 rows in set (0.001 sec)
```

Task 2: Temporal database

Objective: As a normal database user (i.e. not root), create a table AnotherTable as a System-Versioned table. Insert some data, then do some DELETEs, UPDATEs and INSERTs.

Requirements:

- Do not manually create a ROW_START or ROW_END column.
- After all modifications are done, use queries with “FOR SYSTEM_TIME AS OF” to list the before and after values after each modification of the table.
- Every result set should show the state of the table at a specific point in time. Each result sets should include a row only once, showing the state of the row at that specific point in time.

Creating the Table with data and using DELETE, UPDATE and INSERT:

```
MariaDB [ffeilkedatabase]> CREATE TABLE AnotherTable (
    ->     ID INT PRIMARY KEY,
    ->     Value VARCHAR(50)
    -> ) WITH SYSTEM VERSIONING;
Query OK, 0 rows affected (0.020 sec)

MariaDB [ffeilkedatabase]> INSERT INTO AnotherTable (ID, Value) VALUES (1, 'Hyper-Caffeinated Hamster');
Query OK, 1 row affected (0.005 sec)

MariaDB [ffeilkedatabase]> UPDATE AnotherTable SET Value = 'Sentient Espresso Machine' WHERE ID = 1;
Query OK, 1 row affected (0.004 sec)
Rows matched: 1  Changed: 1  Inserted: 1  Warnings: 0

MariaDB [ffeilkedatabase]> INSERT INTO AnotherTable (ID, Value) VALUES (2, 'Invisibility Cloak (Misplaced)');
Query OK, 1 row affected (0.004 sec)

MariaDB [ffeilkedatabase]> DELETE FROM AnotherTable WHERE ID = 2;
Query OK, 1 row affected (0.010 sec)
```

List the before and after values after each modification:

```
MariaDB [ffeilkedatabase]> SELECT Value, ROW_START, ROW_END
    -> FROM AnotherTable
    -> FOR SYSTEM_TIME ALL;
+-------------------------------+----------------------------+----------------------------+
| Value                         | ROW_START                  | ROW_END                    |
+-------------------------------+----------------------------+----------------------------+
| Hyper-Caffeinated Hamster     | 2026-03-20 14:59:52.013136 | 2026-03-20 15:00:03.726658 |
| Sentient Espresso Machine     | 2026-03-20 15:00:03.726658 | 2038-01-19 04:14:07.999999 |
| Invisibility Cloak (Misplaced) | 2026-03-20 15:00:05.693035 | 2026-03-20 15:00:21.132090 |
+-------------------------------+----------------------------+----------------------------+
3 rows in set (0.001 sec)
```

Showing the state of the row at that specific point in time:

```
MariaDB [ffeilkedatabase]> SELECT * FROM AnotherTable
    -> FOR SYSTEM_TIME AS OF '2026-03-20 14:59:52';
Empty set (0.001 sec)

MariaDB [ffeilkedatabase]> SELECT * FROM AnotherTable  FOR SYSTEM_TIME AS OF '2026-03-20 14:59:52.013136';
+----+---------------------------+
| ID | Value                     |
+----+---------------------------+
|  1 | Hyper-Caffeinated Hamster |
+----+---------------------------+
1 row in set (0.001 sec)

MariaDB [ffeilkedatabase]> SELECT * FROM AnotherTable  FOR SYSTEM_TIME AS OF '2026-03-20 15:00:03.726658';
+----+---------------------------+
| ID | Value                     |
+----+---------------------------+
|  1 | Sentient Espresso Machine |
+----+---------------------------+
1 row in set (0.001 sec)

MariaDB [ffeilkedatabase]> SELECT * FROM AnotherTable  FOR SYSTEM_TIME AS OF '2026-03-20 15:00:05.693035';
+----+--------------------------------+
| ID | Value                          |
+----+--------------------------------+
|  1 | Sentient Espresso Machine      |
|  2 | Invisibility Cloak (Misplaced) |
+----+--------------------------------+
2 rows in set (0.001 sec)
```

Task 3: Integrity Constraints

Objective: As a normal database user (i.e. not root), create a teacher table, with numeric attributes for salary, bonus and total.

1. Implement the following integrity constraints: i. The attribute salary must be between 1 000 and 100 000. ii. The attribute total must be the sum of salary and bonus, and calculated automatically.

Insert some test data to verify whether your solution works. Document the output.

Implementation:

```
MariaDB [ffeilkedatabase]> CREATE TABLE TeacherTable (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   salary DECIMAL (10, 2) CHECK (salary BETWEEN 1000 AND 100000),
    ->    bonus DECIMAL (10, 2) DEFAULT 0,
    -> total DECIMAL (10, 2) GENERATED ALWAYS AS (salary + bonus) VIRTUAL
    -> );
```

Verification:

```
MariaDB [ffeilkedatabase]> INSERT INTO TeacherTable (salary, bonus) VALUES (2000, 2000);
Query OK, 1 row affected (0.004 sec)

MariaDB [ffeilkedatabase]> SELECT*FROM TeacherTable;
+----+---------+---------+---------+
| id | salary  | bonus   | total   |
+----+---------+---------+---------+
|  1 | 2000.00 | 2000.00 | 4000.00 |
+----+---------+---------+---------+
1 row in set (0.001 sec)
```

2. Is the table 3NF?

For a table to be in 3NF, every non-prime attribute must be non-transitively dependent on the primary key. In this table, the total depends on salary and bonus rather than just the primary key (ID), therefore, if the total column was stored physically, the table would not be in 3NF. However, we create the value as `VIRTUAL GENERATED` - it is not stored as redundant data on disk, it's calculated dynamically during the query. As such, the physical storage of the table does comply with 3NF.

Task 4: Order of triggers

1. Test the following, and give necessary explanations: a) Create three tables T1, T2, and T3. b) Create a trigger tr12 before insert on T1, insert into T2 a row. c) Create a trigger tr23 after insert on T2, insert into T3 a row. d) Create a trigger tr13 after insert on T1, insert into T3 a row. e) Insert a row into T1, in which order are the triggers fired? Why?

```
MariaDB [ASSIGNMENT4]> CREATE TABLE T1 (
    ->     ID TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ->     MSG VARCHAR(20)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.024 sec)

MariaDB [ASSIGNMENT4]> CREATE TABLE T2 (
    ->     ID TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ->     MSG VARCHAR(20)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.026 sec)

MariaDB [ASSIGNMENT4]> CREATE TABLE T3 (
    ->     ID TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ->     MSG VARCHAR(20),
    ->     SOURCE_TRIGGER VARCHAR(20)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.018 sec)
```

First just creating simple tables to show the workings of triggers

```
MariaDB [ASSIGNMENT4]> DELIMITER //
MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]> CREATE TRIGGER TR12 BEFORE INSERT ON T1 FOR EACH ROW BEGIN INSERT INTO T2 (MSG) VALUES ('INSER
TED_FROM_TR12'); END //
Query OK, 0 rows affected (0.016 sec)

MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]>  CREATE TRIGGER TR23 AFTER INSERT ON T2 FOR EACH ROW BEGIN INSERT INTO T3 (MSG, SOURCE_TRIGGER
) VALUES ('INSERTED_FROM_T2', 'TR23'); END //
Query OK, 0 rows affected (0.014 sec)

MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]>  CREATE TRIGGER TR13 AFTER INSERT ON T1 FOR EACH ROW BEGIN INSERT INTO T3 (MSG, SOURCE_TRIGGER
) VALUES ('INSERTED_FROM_T1', 'TR13'); END //
Query OK, 0 rows affected (0.023 sec)

MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]>  DELIMITER ;
```

Creation of all triggers according to the task

```
MariaDB [ASSIGNMENT4]> SELECT * FROM T1;
+----+-----------------+
| ID | MSG             |
+----+-----------------+
|  1 | INSERTING_VALUE |
+----+-----------------+
1 row in set (0.000 sec)

MariaDB [ASSIGNMENT4]> SELECT * FROM T2;
+----+--------------------+
| ID | MSG                |
+----+--------------------+
|  1 | INSERTED_FROM_TR12 |
+----+--------------------+
1 row in set (0.001 sec)

MariaDB [ASSIGNMENT4]> SELECT * FROM T3 ORDER BY ID ASC;
+----+------------------+----------------+
| ID | MSG              | SOURCE_TRIGGER |
+----+------------------+----------------+
|  1 | INSERTED_FROM_T2 | TR23           |
|  2 | INSERTED_FROM_T1 | TR13           |
+----+------------------+----------------+
2 rows in set (0.000 sec)

MariaDB [ASSIGNMENT4]>
```

After simple insert into T1 we show the content of all tables

The triggers fire in the following order:

`TR12` -> `TR23` -> (Data is INSERTED into `T1`) -> `TR13`

This is because of the cascading triggers.

When we `INSERT` into `T1` the engine first checks for `BEFORE` triggers -> `TR12` fires first.

`TR12` executes `INSERT` into `T2` - trigger execution is synchronous, so MariaDB pauses the original operation `T1` to handle `T2`.

`INSERT` on `T2` fires its own `AFTER` trigger - `TR23`, which inserts a row into `T3`.

After the operations on `T2` (and its triggers) are complete, engine returns to `T1` and the value is insterted into `T1`.

Then the `AFTER INSERT` trigger `TR13` is fired on `T1`, which inserts the row into `T3`.

2. Can there be any unexpected result or deadlock regarding the order of triggers?

Yes, there can be unexpected results and deadlocks. If the triggers form a loop (for example T3 trigger would insert into T1) it would cause infinite recursion. Database engines usually catch this and throw a "maximum recursion depth exceeded" error, which crashes the original transaction.

Also a simple `INSERT` into `T1` can lock `T2` and `T3` with the triggers - could cause performance drops.

Task 5: Pendant DELETE

Objective: Implement the following as a normal database user (i.e. not root):

1. Create two tables with a one-to-many relation, and implement referential integrity. (Create foreign keys on the “many”-table).
2. Add some rows to each table. The child table must have several rows referencing each row of the parent rows.
3. Use triggers to implement the rule known as pendant DELETE (Mullins page 435): A row in the parent table should be deleted if no rows in the child table refer to it any more (when you delete the last row in the child table referencing it).

```
MariaDB [ASSIGNMENT4]> CREATE TABLE PARENT_TABLE (
    ->     ID TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ->     CATEGORY_NAME VARCHAR(30) NOT NULL
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.018 sec)

MariaDB [ASSIGNMENT4]> CREATE TABLE CHILD_TABLE (
    ->     ID TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ->     PARENT_ID TINYINT UNSIGNED NOT NULL,
    ->     ITEM_NAME VARCHAR(30) NOT NULL,
    ->     CONSTRAINT FK_PARENT FOREIGN KEY (PARENT_ID) REFERENCES PARENT_TABLE(ID)
    ->         ON DELETE RESTRICT
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.024 sec)
```

First we again create simple tables: parent table with 1:N relation to child table. We decided to use simple electronics categories and products for the task.

```
MariaDB [ASSIGNMENT4]> INSERT INTO PARENT_TABLE (CATEGORY_NAME) VALUES ('Laptops'), ('Mobile Phones'), ('Tablets');
Query OK, 3 rows affected (0.004 sec)
Records: 3  Duplicates: 0  Warnings: 0

MariaDB [ASSIGNMENT4]> INSERT INTO CHILD_TABLE (PARENT_ID, ITEM_NAME) VALUES (1, 'Thinkpad'), (1, 'MacBook'), (1, 'Le
gion');
Query OK, 3 rows affected (0.004 sec)
Records: 3  Duplicates: 0  Warnings: 0

MariaDB [ASSIGNMENT4]> INSERT INTO CHILD_TABLE (PARENT_ID, ITEM_NAME) VALUES (2, 'Iphone'), (2, 'Galaxy'), (2, 'OnePl
us');
Query OK, 3 rows affected (0.004 sec)
Records: 3  Duplicates: 0  Warnings: 0

MariaDB [ASSIGNMENT4]> INSERT INTO CHILD_TABLE (PARENT_ID, ITEM_NAME) VALUES (3, 'iPad'), (3, 'Samsung Tab'), (3, 'Id
ea Tab');
Query OK, 3 rows affected (0.004 sec)
Records: 3  Duplicates: 0  Warnings: 0

MariaDB [ASSIGNMENT4]> SELECT * FROM PARENT_TABLE;
+----+---------------+
| ID | CATEGORY_NAME |
+----+---------------+
|  1 | Laptops       |
|  2 | Mobile Phones |
|  3 | Tablets       |
+----+---------------+
3 rows in set (0.001 sec)

MariaDB [ASSIGNMENT4]> SELECT * FROM CHILD_TABLE;
+----+-----------+-------------+
| ID | PARENT_ID | ITEM_NAME   |
+----+-----------+-------------+
|  1 |         1 | Thinkpad    |
|  2 |         1 | MacBook     |
|  3 |         1 | Legion      |
|  4 |         2 | Iphone      |
|  5 |         2 | Galaxy      |
|  6 |         2 | OnePlus     |
|  7 |         3 | iPad        |
|  8 |         3 | Samsung Tab |
|  9 |         3 | Idea Tab    |
+----+-----------+-------------+
9 rows in set (0.001 sec)
```

Inserting data into the tables to form 1:N relations.

```
MariaDB [ASSIGNMENT4]> DELIMITER //
MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]> CREATE TRIGGER TRG_PENDANT_DELETE
    -> AFTER DELETE ON CHILD_TABLE
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE CHILD_COUNT TINYINT;
    ->
    ->     -- Count how many children still reference this parent
    ->     SELECT COUNT(*) INTO CHILD_COUNT
    ->     FROM CHILD_TABLE
    ->     WHERE PARENT_ID = OLD.PARENT_ID;
    ->
    ->     -- If no children are left, delete the parent
    ->     IF CHILD_COUNT = 0 THEN
    ->         DELETE FROM PARENT_TABLE WHERE ID = OLD.PARENT_ID;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.015 sec)

MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]> DELIMITER ;
```

Creating a pendant delete trigger - if the child table does not have any values referencing the parent value, delete parent value as well.

```
MariaDB [ASSIGNMENT4]> DELETE FROM CHILD_TABLE WHERE ID = 1;
Query OK, 1 row affected (0.005 sec)

MariaDB [ASSIGNMENT4]> SELECT * FROM PARENT_TABLE;
+----+---------------+
| ID | CATEGORY_NAME |
+----+---------------+
|  1 | Laptops       |
|  2 | Mobile Phones |
|  3 | Tablets       |
+----+---------------+
3 rows in set (0.001 sec)

MariaDB [ASSIGNMENT4]> DELETE FROM CHILD_TABLE WHERE ID = 2;
Query OK, 1 row affected (0.004 sec)

MariaDB [ASSIGNMENT4]> SELECT * FROM PARENT_TABLE;
+----+---------------+
| ID | CATEGORY_NAME |
+----+---------------+
|  1 | Laptops       |
|  2 | Mobile Phones |
|  3 | Tablets       |
+----+---------------+
3 rows in set (0.000 sec)

MariaDB [ASSIGNMENT4]> DELETE FROM CHILD_TABLE WHERE ID = 3;
Query OK, 1 row affected (0.004 sec)

MariaDB [ASSIGNMENT4]> SELECT * FROM PARENT_TABLE;
+----+---------------+
| ID | CATEGORY_NAME |
+----+---------------+
|  2 | Mobile Phones |
|  3 | Tablets       |
+----+---------------+
2 rows in set (0.001 sec)
```

Deletion of one categories items - the category gets deleted after the last product is deleted, as expected.

Explain the point of doing this and why it is difficult compared with other rules on p.435.

1. What is the point of a Pendant DELETE? The purpose of a pendant DELETE is to prevent parent records that no longer serve a purpose - as they do not have any children associated with them. In many business logic scenarios (like a category that only exists to group of active products), a parent record becomes meaningless or clutters the database if it doesn't have any children associated with it. Pendant DELETE automates the cleanup of these empty parent containers.

2. Why is it difficult compared with other rules?

No Native Support: Standard referential integrity rules (like ON DELETE CASCADE or ON DELETE SET NULL that are listed on the page 435) flow "downwards" from parent to child. They are built natively into SQL

engines. Pendant DELETE flows "upwards" (child to parent) and has no native constraint syntax - it must be custom-coded using triggers.

It introduces concurrency and locking risks. For example, if transaction A deletes the last child row and fires the trigger to delete the parent, while at the exact same time transaction B tries to insert a new child row for that same parent, a race condition or deadlock can occur. Transaction B might successfully insert a child pointing to a parent that transaction A just deleted, completely breaking referential integrity. Proper table/row locking is required to do this safely.

Task 6: Concurrency

Objective: An application uses a DBMS to store data about events and participants. Events can have many participants, but the number of spaces should still be limited.

6.1 Normalization and Data Loading

1. Make a normalized, 3NF logical for the data provided in `data.txt` and implement the model in a MariaDB database using InnoDB tables. The model should have an attribute showing the total number of spaces at each event. *(The fastest, and probably also easiest solution is to create a table that temporarily holds all the data. From this table, load data into the normalized tables and afterwards delete the auxiliary table.)*

```
MariaDB [ASSIGNMENT4]> CREATE TABLE EVENT (
    ->     eventId INT UNSIGNED PRIMARY KEY,
    ->     eventTitle VARCHAR(100) NOT NULL,
    ->     eventDate DATETIME NOT NULL,
    ->     totSpaces MEDIUMINT UNSIGNED NOT NULL
    -> ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
Query OK, 0 rows affected (0.019 sec)

MariaDB [ASSIGNMENT4]> CREATE TABLE PARTICIPANT (
    ->     pId INT UNSIGNED PRIMARY KEY,
    ->     sureName VARCHAR(50) NOT NULL,
    ->     givenNames VARCHAR(50) NOT NULL
    -> ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
Query OK, 0 rows affected (0.020 sec)

MariaDB [ASSIGNMENT4]> CREATE TABLE BOOKING (
    ->     eventId INT UNSIGNED,
    ->     pId INT UNSIGNED,
    ->     PRIMARY KEY (eventId, pId),
    ->     CONSTRAINT FK_EVENT FOREIGN KEY (eventId) REFERENCES EVENT(eventId) ON DELETE CASCADE,
    ->     CONSTRAINT FK_PARTICIPANT FOREIGN KEY (pId) REFERENCES PARTICIPANT(pId) ON DELETE CASCADE
    -> ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
Query OK, 0 rows affected (0.024 sec)
```

First we created the tables in 3NF.

```
MariaDB [ASSIGNMENT4]> CREATE TABLE TEMP_DATA (
    ->     eventId INT UNSIGNED,
    ->     eventTitle VARCHAR(100),
    ->     eventDate DATETIME,
    ->     totSpaces MEDIUMINT UNSIGNED,
    ->     pId INT UNSIGNED,
    ->     sureName VARCHAR(50),
    ->     givenNames VARCHAR(50)
    -> ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
Query OK, 0 rows affected (0.023 sec)

MariaDB [ASSIGNMENT4]> LOAD DATA LOCAL INFILE '/home/soukup/Documents/Assignment4/data.txt'
    -> INTO TABLE TEMP_DATA
    -> CHARACTER SET utf8mb4
    -> FIELDS TERMINATED BY ';'
    -> LINES TERMINATED BY '\n'
    -> (eventId, eventTitle, eventDate, totSpaces, pId, sureName, givenNames);
Query OK, 281396 rows affected (1.221 sec)
Records: 281396  Deleted: 0  Skipped: 0  Warnings: 0
```

Then we created a dummy table to load all the data from the `data.txt` file into. We had to use `CHARACTER SET utf8mb4` while loading to ensure we load the characters as intended.

```
MariaDB [ASSIGNMENT4]> SELECT DISTINCT eventTitle from TEMP_DATA;
+----------------------------+
| eventTitle                 |
+----------------------------+
| Stolzekleiven opp          |
| Knarvikmila                |
| Led Zeppelin at Koengen    |
| The 7-mountain hike        |
| Ulriken down               |
| AHA at Koengen             |
| Bergen City Maraton        |
| Stones at Koengen          |
| Lyderhorn opp              |
| ABBA at Koengen            |
| Kygo at Brann Stadion      |
| Pink Floyd at Koengen      |
| Guided tour to Løvstakken  |
+----------------------------+
13 rows in set (0.195 sec)
```

Verification select to see if the data was loaded correctly as well as the characters.

```
MariaDB [ASSIGNMENT4]> INSERT INTO EVENT (eventId, eventTitle, eventDate, totSpaces)
    -> SELECT DISTINCT eventId, eventTitle, eventDate, totSpaces
    -> FROM TEMP_DATA;
Query OK, 13 rows affected (0.378 sec)
Records: 13  Duplicates: 0  Warnings: 0

MariaDB [ASSIGNMENT4]> INSERT INTO PARTICIPANT (pId, sureName, givenNames)
    -> SELECT DISTINCT pId, sureName, givenNames
    -> FROM TEMP_DATA;
Query OK, 274738 rows affected (4.658 sec)
Records: 274738  Duplicates: 0  Warnings: 0

MariaDB [ASSIGNMENT4]> INSERT INTO BOOKING (eventId, pId)
    -> SELECT DISTINCT eventId, pId
    -> FROM TEMP_DATA;
Query OK, 281396 rows affected (2.689 sec)
Records: 281396  Duplicates: 0  Warnings: 0
```

Inserting the data into their respective tables.

```
MariaDB [ASSIGNMENT4]> SELECT * FROM EVENT;
+---------+---------------------------+---------------------+-----------+--------------+
| eventId | eventTitle                | eventDate           | totSpaces | bookedSpaces |
+---------+---------------------------+---------------------+-----------+--------------+
|       1 | Ulriken down              | 2026-07-01 10:00:00 |     15000 |        10572 |
|       2 | The 7-mountain hike       | 2026-05-23 09:00:00 |     32000 |        22400 |
|       3 | Pink Floyd at Koengen     | 2026-06-16 08:00:00 |     25000 |        17388 |
|       4 | AHA at Koengen            | 2026-08-05 07:00:00 |     23000 |        16028 |
|       5 | Guided tour to Løvstakken | 2026-06-03 09:00:00 |       230 |            1 |
|       6 | ABBA at Koengen           | 2026-08-10 08:00:00 |     22000 |        15239 |
|       7 | Led Zeppelin at Koengen   | 2026-07-12 19:30:00 |     25000 |        17451 |
|       8 | Bergen City Maraton       | 2026-05-12 08:00:00 |     78000 |        55024 |
|       9 | Knarvikmila               | 2026-06-23 08:30:00 |     53000 |        36916 |
|      10 | Lyderhorn opp             | 2026-08-12 09:00:00 |     34000 |        23906 |
|      11 | Stones at Koengen         | 2026-06-13 09:00:00 |     24000 |        16864 |
|      12 | Kygo at Brann Stadion     | 2026-07-15 10:20:00 |     21000 |        14753 |
|      13 | Stolzekleiven opp         | 2026-09-12 10:00:00 |     50000 |        34854 |
+---------+---------------------------+---------------------+-----------+--------------+
13 rows in set (0.001 sec)

MariaDB [ASSIGNMENT4]> select count(*) from PARTICIPANT;
+----------+
| count(*) |
+----------+
|   274738 |
+----------+
1 row in set (0.091 sec)

MariaDB [ASSIGNMENT4]> select count(*) from BOOKING;
+----------+
| count(*) |
+----------+
|   281396 |
+----------+
1 row in set (0.048 sec)

MariaDB [ASSIGNMENT4]>
```

Verification selects to make sure we loaded the data correctly into the tables.

After the tables vere verified we deleted the temporary dummy table.

6.2 Validation Queries

2. Before continuing, you must confirm that your model is working, and that the data got correctly loaded into the database. Formulate the queries below and test your database.

i. Find the maximum number of events that a single participant will attend. (Avoid ORDER BY)

```
MariaDB [ASSIGNMENT4]> SELECT MAX(arr.c) AS `MAX(arr.c)`
    -> FROM (
    ->     SELECT COUNT(eventId) AS c
    ->     FROM BOOKING
    ->     GROUP BY pId
    -> ) AS arr;
+------------+
| MAX(arr.c) |
+------------+
|          4 |
+------------+
1 row in set (0.132 sec)
```

ii. List names of the participants that attend the largest number of events. (Do not use LIMIT, Avoid ORDER BY)

```
MariaDB [ASSIGNMENT4]> SELECT p.pId, max(p.givenNames), max(p.sureName)
    -> FROM PARTICIPANT p
    -> JOIN BOOKING b ON p.pId = b.pId
    -> GROUP BY p.pId
    -> HAVING COUNT(b.eventId) = (
    ->     SELECT MAX(arr.c)
    ->     FROM (
    ->         SELECT COUNT(eventId) AS c
    ->         FROM BOOKING
    ->         GROUP BY pId
    ->     ) AS arr
    -> );
+---------+-------------------+-----------------+
| pId     | max(p.givenNames) | max(p.sureName) |
+---------+-------------------+-----------------+
| 4359525 | Astrid            | Gjerstad        |
| 4415521 | Une               | Hov             |
| 4504833 | Liselotte         | Martinussen     |
+---------+-------------------+-----------------+
3 rows in set (0.538 sec)

MariaDB [ASSIGNMENT4]>
```

After the review this was done we changed it to use `max()` instead of our initial `group by` query with the data.

iii. What events are attended by Ludvig Rustad?

```
MariaDB [ASSIGNMENT4]> SELECT e.eventTitle
    -> FROM EVENT e
    -> JOIN BOOKING b ON e.eventId = b.eventId
    -> JOIN PARTICIPANT p ON b.pId = p.pId
    -> WHERE p.givenNames = 'Ludvig' AND p.sureName = 'Rustad';
+-------------------+
| eventTitle        |
+-------------------+
| Knarvikmila       |
| Lyderhorn opp     |
| Stones at Koengen |
+-------------------+
3 rows in set (0.406 sec)
```

iv. List participants that attend exactly 3 events.

```
MariaDB [ASSIGNMENT4]> SELECT p.pId, p.givenNames, p.sureName
    -> FROM PARTICIPANT p
    -> JOIN BOOKING b ON p.pId = b.pId
    -> GROUP BY p.pId, p.givenNames, p.sureName
    -> HAVING COUNT(b.eventId) = 3;
+---------+------------+---------------+
| pId     | givenNames | sureName      |
+---------+------------+---------------+
|   25113 | Zoulikha   | Wilhelmsen    |
|   72717 | Ulf        | Erstad        |
|  154447 | Asbjørg    | Malik         |
|  215857 | Leon       | Heimdal       |
|  366353 | Sigurd     | Singh         |
|  389501 | Yoon       | Sanden        |
|  394085 | Atle       | Foss          |

| 4413626 | Zeinab     | Nordbø        |
| 4601207 | Nicklas    | Sharma        |
| 4618876 | Matilde    | Rosvold       |
| 4681795 | Thea       | Osmundsen     |
| 4724486 | Jose       | Ommundsen     |
| 4908919 | Sam        | Hansen        |
| 4910264 | Janita     | Akselsen      |
| 4948135 | Pranee     | Engan         |
| 4993299 | Tahir      | Amundsen      |
| 5082693 | Sally      | Nordahl       |
+---------+------------+---------------+
89 rows in set (3.639 sec)

MariaDB [ASSIGNMENT4]>
```

v. What events do they attend, the participants that attend 3 events? (Your query should not refer explicitly to any events, nor eventId equal to 5)

```
MariaDB [ASSIGNMENT4]> SELECT DISTINCT e.eventId, e.eventTitle
    -> FROM EVENT e
    -> JOIN BOOKING b ON e.eventId = b.eventId
    -> WHERE b.pId IN (
    ->     SELECT pId
    ->     FROM BOOKING
    ->     GROUP BY pId
    ->     HAVING COUNT(eventId) = 3
    -> );
+---------+-------------------------+
| eventId | eventTitle              |
+---------+-------------------------+
|       1 | Ulriken down            |
|       2 | The 7-mountain hike     |
|       3 | Pink Floyd at Koengen   |
|       4 | AHA at Koengen          |
|       6 | ABBA at Koengen         |
|       7 | Led Zeppelin at Koengen |
|       8 | Bergen City Maraton     |
|       9 | Knarvikmila             |
|      10 | Lyderhorn opp           |
|      11 | Stones at Koengen       |
|      12 | Kygo at Brann Stadion   |
|      13 | Stolzekleiven opp       |
+---------+-------------------------+
12 rows in set (0.051 sec)
```

vi. Are there any events that is not attended by anybody that attends three events? (Your query should not refer explicitly any of the events)

```
MariaDB [ASSIGNMENT4]> SELECT eventId, eventTitle
    -> FROM EVENT
    -> WHERE eventId NOT IN (
    ->     SELECT eventId
    ->     FROM BOOKING
    ->     WHERE pId IN (
    ->         SELECT pId
    ->         FROM BOOKING
    ->         GROUP BY pId
    ->         HAVING COUNT(eventId) = 3
    ->     )
    -> );
+---------+----------------------------+
| eventId | eventTitle                 |
+---------+----------------------------+
|       5 | Guided tour to Løvstakken  |
+---------+----------------------------+
1 row in set (0.114 sec)

MariaDB [ASSIGNMENT4]>
```

vii. Are there any events that is attended only by the participants that attend only one event? (Your query should not refer explicitly any of the events)

```
MariaDB [ASSIGNMENT4]> SELECT DISTINCT e.eventId, e.eventTitle
    -> FROM EVENT e
    -> JOIN BOOKING b ON e.eventId = b.eventId
    -> WHERE e.eventId NOT IN (
    ->     -- events attended by people who go to more than 1 event
    ->     SELECT eventId
    ->     FROM BOOKING
    ->     WHERE pId IN (
    ->         SELECT pId
    ->         FROM BOOKING
    ->         GROUP BY pId
    ->         HAVING COUNT(eventId) > 1
    ->     )
    -> );
+---------+-----------------------------+
| eventId | eventTitle                  |
+---------+-----------------------------+
|       5 | Guided tour to LÃ¸vstakken   |
+---------+-----------------------------+
1 row in set (0.121 sec)
```

The first query that we discussed was wrong at the time.

```
MariaDB [ASSIGNMENT4]> SELECT eventId, eventTitle
    -> FROM EVENT
    -> WHERE eventId IN (
    ->     -- The event is attended by people who attend exactly 1 event
    ->     SELECT eventId
    ->     FROM BOOKING
    ->     WHERE pId IN (
    ->         SELECT pId
    ->         FROM BOOKING
    ->         GROUP BY pId
    ->         HAVING COUNT(eventId) = 1
    ->     )
    -> )
    -> AND eventId NOT IN (
    ->     -- The event is not attended by people who attend more than 1 event
    ->     SELECT eventId
    ->     FROM BOOKING
    ->     WHERE pId IN (
    ->         SELECT pId
    ->         FROM BOOKING
    ->         GROUP BY pId
    ->         HAVING COUNT(eventId) > 1
    ->     )
    -> );
+---------+---------------------------+
| eventId | eventTitle                |
+---------+---------------------------+
|       5 | Guided tour to Løvstakken |
+---------+---------------------------+
1 row in set (0.366 sec)

MariaDB [ASSIGNMENT4]>
```

Query fixed with the `AND` clause

6.3 Stored Procedure takeSpace

3. Make a stored procedure `takeSpace` that will book a space for an existing participant at an event.

- If there are available spaces at the event, book a space for the participant.
- If the event is sold out, do nothing.
- The procedure should be accessible by concurrent users, i.e. locking is required with a 3NF model.
- The stored procedure should not modify column totSpaces of the table Event.

```
MariaDB [ASSIGNMENT4]> DELIMITER //
MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]> CREATE PROCEDURE TAKESPACE(IN IN_PID INT UNSIGNED, IN IN_EVENTID INT UNSIGNED)
    -> BEGIN
    ->     DECLARE V_TOTALSPACES MEDIUMINT UNSIGNED;
    ->     DECLARE V_BOOKEDSPACES MEDIUMINT UNSIGNED;
    ->
    ->     -- Start a new transaction to manage locks
    ->     START TRANSACTION;
    ->
    ->     -- 1) Lock the specific EVENT row to prevent other concurrent bookings from other users for the chosen event. All other transactions attempting to book this event will wait here until this transaction is committed
    ->     SELECT totSpaces INTO V_TOTALSPACES
    ->     FROM EVENT
    ->     WHERE eventId = IN_EVENTID
    ->     FOR UPDATE;
    ->
    ->     -- 2) Count how many people are currently booked for this event
    ->     SELECT COUNT(*) INTO V_BOOKEDSPACES
    ->     FROM BOOKING
    ->     WHERE eventId = IN_EVENTID;
    ->
    ->     -- 3) Check if there is room available to make the booking
    ->     IF V_BOOKEDSPACES < V_TOTALSPACES THEN
    ->         -- Insert the booking. IGNORE is used so it doesn't crash if the user is already booked (it would throw a fatal constraint violation error and crash the procedure, abort transaction and throw an error)
    ->         INSERT IGNORE INTO BOOKING (eventId, pId) VALUES (IN_EVENTID, IN_PID);
    ->     END IF;
    ->
    ->     -- 4) Commit the transaction - releases the FOR UPDATE lock
    ->     COMMIT;
    -> END //
Query OK, 0 rows affected (0.015 sec)

MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]> DELIMITER ;
```

The first try at the procedure - we discussed that it would not lock the correct table but in the end it also does, at least from the lecturers comment we got after the lab.

```
MariaDB [ASSIGNMENT4]> DELIMITER //
MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]> CREATE PROCEDURE TAKESPACE(IN IN_PID INT UNSIGNED, IN IN_EVENTID INT UNSIGNED)
    -> BEGIN
    ->     DECLARE V_TOTALSPACES MEDIUMINT UNSIGNED;
    ->     DECLARE V_BOOKEDSPACES MEDIUMINT UNSIGNED;
    ->
    ->     -- Start the transaction
    ->     START TRANSACTION;
    ->
    ->     -- 1) Lock the BOOKING table first for this specific event
    ->     SELECT COUNT(*) INTO V_BOOKEDSPACES
    ->     FROM BOOKING
    ->     WHERE eventId = IN_EVENTID
    ->     FOR UPDATE;
    ->
    ->     -- 2) Read the total allowed spaces from the EVENT table
    ->     SELECT totSpaces INTO V_TOTALSPACES
    ->     FROM EVENT
    ->     WHERE eventId = IN_EVENTID;
    ->
    ->     -- 3) Check capacity and insert if there is room
    ->     IF V_BOOKEDSPACES < V_TOTALSPACES THEN
    ->         INSERT IGNORE INTO BOOKING (eventId, pId) VALUES (IN_EVENTID, IN_PID);
    ->     END IF;
    ->
    ->     -- 4) Commit to release the locks
    ->     COMMIT;
    -> END //
Query OK, 0 rows affected (0.009 sec)
```

Second try to lock the table correctly.

```
MariaDB [ASSIGNMENT4]> INSERT INTO EVENT (eventId, eventTitle, eventDate, totSpaces)
    -> VALUES (999, 'TINY CONCERT', '2026-12-31 20:00:00', 1);
Query OK, 1 row affected (0.005 sec)
```

Create new concert event.

```
MariaDB [ASSIGNMENT4]> INSERT INTO PARTICIPANT (pId, sureName, givenNames) VALUES (10000, 'Smith', 'John');
Query OK, 1 row affected (0.003 sec)

MariaDB [ASSIGNMENT4]> INSERT INTO PARTICIPANT (pId, sureName, givenNames) VALUES (10001, 'Doe', 'Jane');
Query OK, 1 row affected (0.004 sec)

MariaDB [ASSIGNMENT4]> CALL TAKESPACE(10000, 999);
Query OK, 3 rows affected (0.007 sec)

MariaDB [ASSIGNMENT4]> SELECT * FROM BOOKING WHERE eventId = 999;
+---------+-------+
| eventId | pId   |
+---------+-------+
|     999 | 10000 |
+---------+-------+
1 row in set (0.001 sec)

MariaDB [ASSIGNMENT4]> CALL TAKESPACE(10001, 999);
Query OK, 2 rows affected (0.001 sec)

MariaDB [ASSIGNMENT4]> SELECT * FROM BOOKING WHERE eventId = 999;
+---------+-------+
| eventId | pId   |
+---------+-------+
|     999 | 10000 |
+---------+-------+
1 row in set (0.001 sec)
```

Testing the stored procedure. This works, but for actual robust testing we would need to try multiple concurrent queries.

6.4 Profiling

4. Start profiling and check the time consumption used by the stored procedure when booking a space at an event. What takes most time?

- Discuss how multiple concurrent bookings for the same event would influence the time consumption.

```
MariaDB [ASSIGNMENT4]> INSERT INTO PARTICIPANT (pId, sureName, givenNames) VALUES (10002, 'Smith', 'Alice');
Query OK, 1 row affected (0.006 sec)

MariaDB [ASSIGNMENT4]> SET profiling = 1;
Query OK, 0 rows affected (0.003 sec)

MariaDB [ASSIGNMENT4]> CALL TAKESPACE(10002, 13);
Query OK, 3 rows affected (0.031 sec)

MariaDB [ASSIGNMENT4]> SHOW PROFILES;
+----------+------------+-----------------------------------------------------------------------------------------------------------+
| Query_ID | Duration   | Query                                                                                                     |
+----------+------------+-----------------------------------------------------------------------------------------------------------+
|        1 | 0.00003820 | START TRANSACTION                                                                                         |
|        2 | 0.00048386 | SELECT totSpaces INTO V_TOTALSPACES
    FROM EVENT
    WHERE eventId = IN_EVENTID
    FOR UPDATE |
|        3 | 0.02558920 | SELECT COUNT(*) INTO V_BOOKEDSPACES
    FROM BOOKING
    WHERE eventId = IN_EVENTID               |
|        4 | 0.00027996 | INSERT IGNORE INTO BOOKING (eventId, pId) VALUES (IN_EVENTID, IN_PID)                                     |
|        5 | 0.00184536 | COMMIT                                                                                                    |
+----------+------------+-----------------------------------------------------------------------------------------------------------+
5 rows in set (0.002 sec)

MariaDB [ASSIGNMENT4]> SHOW PROFILE FOR QUERY 1;
+----------------------------+----------+
| Status                     | Duration |
+----------------------------+----------+
| continuing inside routine | 0.000016 |
| Query end                  | 0.000005 |
| closing tables             | 0.000004 |
| Query end                  | 0.000004 |
| Starting cleanup           | 0.000009 |
+----------------------------+----------+
5 rows in set (0.001 sec)
```

Setup and following profiling queries

```
MariaDB [ASSIGNMENT4]> SHOW PROFILE FOR QUERY 2;
+----------------------------+----------+
| Status                     | Duration |
+----------------------------+----------+
| continuing inside routine | 0.000019 |
| checking permissions       | 0.000009 |
| Opening tables             | 0.000030 |
| After opening tables       | 0.000012 |
| System lock                | 0.000010 |
| table lock                 | 0.000054 |
| init                       | 0.000037 |
| Optimizing                 | 0.000026 |
| Statistics                 | 0.000085 |
| Preparing                  | 0.000059 |
| Unlocking tables           | 0.000009 |
| Preparing                  | 0.000027 |
| Executing                  | 0.000006 |
| Sending data               | 0.000012 |
| End of update loop         | 0.000013 |
| Query end                  | 0.000023 |
| Commit                     | 0.000008 |
| Query end                  | 0.000006 |
| closing tables             | 0.000006 |
| Unlocking tables           | 0.000004 |
| closing tables             | 0.000008 |
| Query end                  | 0.000007 |
| Starting cleanup           | 0.000014 |
+----------------------------+----------+
23 rows in set (0.000 sec)
```

```
MariaDB [ASSIGNMENT4]> SHOW PROFILE FOR QUERY 3;
+----------------------------+----------+
| Status                     | Duration |
+----------------------------+----------+
| continuing inside routine | 0.000017 |
| checking permissions       | 0.000007 |
| Opening tables             | 0.000019 |
| After opening tables       | 0.000006 |
| System lock                | 0.000006 |
| table lock                 | 0.000008 |
| init                       | 0.000025 |
| Optimizing                 | 0.000016 |
| Statistics                 | 0.000120 |
| Preparing                  | 0.000042 |
| Executing                  | 0.000005 |
| Sending data               | 0.025245 |
| End of update loop         | 0.000018 |
| Query end                  | 0.000006 |
| Commit                     | 0.000006 |
| Query end                  | 0.000006 |
| closing tables             | 0.000006 |
| Unlocking tables           | 0.000006 |
| closing tables             | 0.000007 |
| Query end                  | 0.000006 |
| Starting cleanup           | 0.000013 |
+----------------------------+----------+
21 rows in set (0.001 sec)
```

```
MariaDB [ASSIGNMENT4]> SHOW PROFILE FOR QUERY 4;
+----------------------------+----------+
| Status                     | Duration |
+----------------------------+----------+
| continuing inside routine | 0.000016 |
| checking permissions       | 0.000006 |
| Opening tables             | 0.000018 |
| After opening tables       | 0.000006 |
| System lock                | 0.000006 |
| table lock                 | 0.000007 |
| init for update            | 0.000021 |
| Update                     | 0.000146 |
| End of update loop         | 0.000009 |
| Query end                  | 0.000005 |
| Commit                     | 0.000006 |
| Query end                  | 0.000005 |
| closing tables             | 0.000005 |
| Unlocking tables           | 0.000005 |
| closing tables             | 0.000006 |
| Query end                  | 0.000005 |
| Starting cleanup           | 0.000008 |
+----------------------------+----------+
17 rows in set (0.001 sec)
```

```
MariaDB [ASSIGNMENT4]> SHOW PROFILE FOR QUERY 5;
+----------------------------+----------+
| Status                     | Duration |
+----------------------------+----------+
| continuing inside routine | 0.000009 |
| Commit                     | 0.001790 |
| Starting cleanup           | 0.000017 |
| Query end                  | 0.000008 |
| closing tables             | 0.000005 |
| Query end                  | 0.000005 |
| Starting cleanup           | 0.000011 |
+----------------------------+----------+
7 rows in set (0.001 sec)
```

What takes most time?

Operations involving disk I/O take the most time, sending the data, updating and commit.

*** Discuss how multiple concurrent bookings for the same event would influence the time consumption.**

Because our procedure uses a pessimistic lock `SELECT ... FOR UPDATE`, it prevents parallel processing for the same event row.

If 1,000 users try to book a space at the same event at the same time, the first user locks the row. The rest of the users are placed in a waiting queue.

User 2 cannot start checking the available spaces until User 1 finishes their entire process and commits (releasing the lock).

Therefore, the time consumption for concurrent users will degrade linearly. The 100th user's transaction will take significantly longer to complete than the 1st user's because their total time includes the execution time of the 99 transactions ahead of them in the lock queue.

6.5 Denormalisation

5. Can you advice a denormalisation scheme that would reduce the time consumption?

- Implement the new model, and update `takeSpace` to take the new model into account. Try to avoid the need for explicit locks in the updated version of `takeSpace`.
- Use profiling and measure the time consumption when booking a space at an event with the new version `takeSpace`.

```
MariaDB [ASSIGNMENT4]> ALTER TABLE EVENT ADD COLUMN bookedSpaces MEDIUMINT UNSIGNED NOT NULL DEFAULT 0;
Query OK, 0 rows affected (0.030 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [ASSIGNMENT4]> UPDATE EVENT e
    -> SET e.bookedSpaces = (
    ->     SELECT COUNT(*)
    ->     FROM BOOKING b
    ->     WHERE b.eventId = e.eventId
    -> );
Query OK, 14 rows affected (0.233 sec)
Rows matched: 14  Changed: 14  Warnings: 0
```

Updating the EVENT table with new column `booked spaces` that counts the number of bookings for the event.

```
MariaDB [ASSIGNMENT4]> DROP PROCEDURE IF EXISTS TAKESPACE //
Query OK, 0 rows affected (0.017 sec)

MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]> CREATE PROCEDURE TAKESPACE(IN IN_PID INT UNSIGNED, IN IN_EVENTID INT UNSIGNED)
    -> BEGIN
    ->     -- 1) Attempt to get a space; this statement is thread-safe, so we don't need ecplicit locks
    ->     UPDATE EVENT
    ->     SET bookedSpaces = bookedSpaces + 1
    ->     WHERE eventId = IN_EVENTID AND bookedSpaces < totSpaces;
    ->
    ->     -- 2) Check if the UPDATE actually changed a row using ROW_COUNT()
    ->     IF ROW_COUNT() > 0 THEN
    ->         -- user got a spot, now we insert the booking
    ->         INSERT IGNORE INTO BOOKING (eventId, pId) VALUES (IN_EVENTID, IN_PID);
    ->     END IF;
    ->
    -> END //
Query OK, 0 rows affected (0.013 sec)

MariaDB [ASSIGNMENT4]>
MariaDB [ASSIGNMENT4]> DELIMITER ;
```

Updating the procedure to now work with the edited table.

```
MariaDB [ASSIGNMENT4]> INSERT INTO PARTICIPANT (pId, sureName, givenNames) VALUES (10003, 'Walker', 'Alan');
Query OK, 1 row affected (0.006 sec)

MariaDB [ASSIGNMENT4]> SET profiling = 1;
Query OK, 0 rows affected (0.000 sec)

MariaDB [ASSIGNMENT4]> CALL TAKESPACE(10003, 13);
Query OK, 2 rows affected (0.007 sec)

MariaDB [ASSIGNMENT4]> SHOW PROFILES;
+----------+------------+-----------------------------------------------------------------------------------------------------------------------------------+
| Query_ID | Duration   | Query                                                                                                                             |
+----------+------------+-----------------------------------------------------------------------------------------------------------------------------------+
|        1 | 0.00003820 | START TRANSACTION                                                                                                                 |
|        2 | 0.00048386 | SELECT totSpaces INTO V_TOTALSPACES
    FROM EVENT
    WHERE eventId = IN_EVENTID
    FOR UPDATE                |
|        3 | 0.02558920 | SELECT COUNT(*) INTO V_BOOKEDSPACES
    FROM BOOKING
    WHERE eventId = IN_EVENTID                              |
|        4 | 0.00027996 | INSERT IGNORE INTO BOOKING (eventId, pId) VALUES (IN_EVENTID, IN_PID)                                                             |
|        5 | 0.00184536 | COMMIT                                                                                                                            |
|        6 | 0.00203808 | UPDATE EVENT
    SET bookedSpaces = bookedSpaces + 1
    WHERE eventId = IN_EVENTID AND bookedSpaces < totSpaces |
|        7 | 0.00174927 | INSERT IGNORE INTO BOOKING (eventId, pId) VALUES (IN_EVENTID, IN_PID)                                                             |
+----------+------------+-----------------------------------------------------------------------------------------------------------------------------------+
7 rows in set (0.000 sec)

MariaDB [ASSIGNMENT4]> set profiling = 0;
Query OK, 0 rows affected (0.000 sec)
```

Profiling new data and showing the results.

```
MariaDB [ASSIGNMENT4]> SHOW PROFILE FOR QUERY 6;
+----------------------------+----------+
| Status                     | Duration |
+----------------------------+----------+
| continuing inside routine  | 0.000017 |
| checking permissions       | 0.000005 |
| Opening tables             | 0.000025 |
| After opening tables       | 0.000005 |
| System lock                | 0.000004 |
| table lock                 | 0.000007 |
| init for update            | 0.000082 |
| Updating                   | 0.000084 |
| End of update loop         | 0.000009 |
| Query end                  | 0.000004 |
| Commit                     | 0.001762 |
| Query end                  | 0.000004 |
| closing tables             | 0.000003 |
| Unlocking tables           | 0.000002 |
| closing tables             | 0.000002 |
| Query end                  | 0.000003 |
| Starting cleanup           | 0.000018 |
+----------------------------+----------+
17 rows in set (0.001 sec)
```

```
MariaDB [ASSIGNMENT4]> SHOW PROFILE FOR QUERY 7;
+----------------------------+----------+
| Status                     | Duration |
+----------------------------+----------+
| continuing inside routine | 0.000008 |
| checking permissions       | 0.000004 |
| Opening tables             | 0.000010 |
| After opening tables       | 0.000003 |
| System lock                | 0.000003 |
| table lock                 | 0.000004 |
| init for update            | 0.000011 |
| Update                     | 0.000072 |
| End of update loop         | 0.000005 |
| Query end                  | 0.000003 |
| Commit                     | 0.001610 |
| Query end                  | 0.000004 |
| closing tables             | 0.000002 |
| Unlocking tables           | 0.000002 |
| closing tables             | 0.000002 |
| Query end                  | 0.000003 |
| Starting cleanup           | 0.000004 |
+----------------------------+----------+
17 rows in set (0.001 sec)
```

The longest time now take the commits. Our old method's SELECT COUNT(*) took 0.0255 seconds. The new method completed the entire atomic UPDATE in 0.0020 seconds.

We now don't have any explicit locks - there is no `START TRANSACTION` or `FOR UPDATE` in queries 6 and 7. By moving the logic to a single UPDATE statement, the database engine handles the concurrency automatically.